SO-WebServer: Technical Document

1. Introduction

This document provides a comprehensive technical overview of the project, detailing its design decisions, and implementation strategies. It aims to serve as a reference for understanding the system's functionality, and the reasoning behind key technical choices. Please read the design document before reading this report in order to completely comprehend how the server works and why our choices were made.

The system is designed to accept HTTP requests (or HTTPS in our scenario, as we will see with more detail further forward), having a master process that is responsible for accepting the incoming connections and distributing them to worker processes that have individual thread pools in order to handle multiple concurrent requests.

As the main focus of this project is handling concurrency correctly, it is essential to ensure consistency and reliability of the stored (shared) data that can be achieved through the utilization of POSIX semaphores.

It is also important to provide a user-friendly interface for analyzing information such as the server statistics and ensure that our build allows scalability and is able to handle heavy server traffic.

Please note that in order to complete this project successfully, we consulted the given support documentation and several online tools to gather information and learn necessary concepts, as well as debugging errors, such as:

- Online Coding Discussion Forums, such as StackOverflow, for example;
- LLM models, to learn concepts, help in debugging, generate front-end code (stats.html) and translate phrases to English, in order to write the necessary documentation, such as this report;

2. Implementation Details

Before we approach the required features implementation details it is important to know how the server is set up and how it shuts down.

The server initialization is a responsibility of the master process that has a master_init function, which creates the shared memory, semaphores and cache for workers, then the server socket and finally the workers themselves, using the variables available on the server config file. All of this is achieved by calling the appropriate functions. An essential piece of this puzzle is a variable available on the shared memory called keep_running that will be used in almost every while loop in our code, in order to achieve a graceful server shutdown.

After it is complete, we can move to the master accept loop to accept incoming connections.

Once the program is interrupted (by CTRL+C or CTRL+Z), the server shuts down, but it is not that simple.
The signal (SIGINT) is intercepted by a function called master_signal_handler, that sets keep_running to 0, so that every while loop stops and each running process can start its cleanup in order to avoid memory leaks and achieve the graceful shutdown we aim for. This includes:

- Closing all open sockets to ensure no client connections remain active;
- Releasing allocated shared memory segments;
- Destroying semaphores used from synchronizing access to shared resources;
- Terminating worker threads gracefully, ensuring that any ongoing request processing completes without leaving resources in an unexpected state;
- Wait for every worker process to finish (waitpid) in order to avoid zombie processes and ensure that all child processes have terminated cleanly before the master process exists.

2.1. Connection Queue

Let's begin with the theory behind accepting connections.

The master process accepts incoming connections and delegates the request handling responsibility to the worker. The worker then proceeds to send a response and the master awaits for the next connection. Later we will detail how we handle concurrent connections using thread pools.

Our main struggle developing this feature was the inter-process communication and how File Descriptors could be properly sent to the worker processes.

In our initial approach, we attempted to pass the integer value corresponding to the client's file descriptor to the worker through shared memory. However, this method proved ineffective due to the way file descriptor tables operate.

Each process maintains its own file descriptor table. Although worker processes are created via fork and therefore inherit the master process's file descriptor table, this inheritance only reflects the state of the table at the moment the fork occurs. Since all workers are forked **before** any client connection is accepted, the accepted socket file descriptors do not exist in the workers' inherited tables. As a result, even though the workers share the same initial descriptor table structure, they do *not* have access to file descriptors created after the fork, making it impossible for them to use the integer value provided through shared memory to access the client connection.

To address this issue, we implemented a communication channel between the master and worker processes using Unix Domain Sockets (UDS). This approach allows the master process to safely transmit the accepted client file descriptors to the workers after the connection has been established. By sending the file descriptors through UDS, each worker receives a valid reference to the client connection, enabling proper request handling by using its thread pool to process the necessary tasks.

2.2. Thread Pool Management

As mentioned in the previous section, after a worker receives a valid reference to the client connection, it is now the worker's job to handle that request using one of the threads in its thread pool.

When creating each worker, right at the beginning each one also creates its own private thread pool. By default, each one of these pools has a total of 10 threads. When a request is sent to a worker, that worker enqueues that connection to the thread pool, so that an available thread can handle that request. If for whatever reason there are no available threads, the server will send a status 503 message, meaning that server is unavailable at that moment.

A thread pool is better than creating a thread every time one is needed because that way we can reuse threads once they are done processing a request. Worker threads are actively attempting to dequeue a request from the queue stored in each pool.

That individual worker thread will then process that request, by parsing it and taking the necessary measures to send back a response to the client. Those measures could be building a brand new response or using a pre-existing one that is stored in cache*. Once they are done processing it, the thread goes back to its loop of attempting to dequeue whatever request comes next.

*The details of the caching mechanism are present in a later section of this document.

2.3. Shared Statistics

The shared statistics feature allows the server to collect and maintain information about its operation, including the following parameters:

- Total requests served;
- Bytes transferred;
- Requests per HTTP status code (200, 206, 403, 404, 500, 503);
- Active connections count;
- Average response time (in milliseconds);

This data is useful for monitoring, debugging and performance analysis of the server.

Statistics are stored in a shared memory accessible by all worker processes and the master process. In order to avoid race conditions and to ensure data's consistency and integrity, we use a mutex semaphore. Each operation acquires the mutex before modifying or accessing the statistics and releases it immediately afterwards.

It is the worker's responsibility to update the statistics after a request is processed and the master process reads and prints the statistics periodically (every 30 seconds). Later we will detail how we give the user a way to keep track of these statistics in real-time.

2.4 Thread-Safe File Cache

The file caching feature allows the server to store the least recently used (LRU) HTTP responses and use them when the client gives the corresponding request.

The existence of this cache saves time for the server to send a response, since it doesn't need to build a new one every time it receives a request.

Each worker has its own cache, which by default stores a total of 10MB of built responses. This cache is created in the master process, and gets copied over to each worker when forking.

When a worker receives a request, it parses it as normal, and then checks if that response exists in the cache:

- If it does, it will get that response and send it immediately to the client, thus avoiding the need to build a completely new response for the same request and saving time;
- If it doesn't, it will build a new response from scratch, save that response to the cache and then send it to the client, making that already built response available for future use.

Whenever a worker adds a newly built response to the cache, the cache checks whether there's enough space to store it. If so, the response is added normally and becomes available to subsequent requests. If not, the cache removes the oldest entry to make room for the new one. That removed response will only re-enter the cache if a client requests it again.

2.5. Thread-Safe Logging

The thread-safe logging feature enables the server to log whatever it needs at any time, in order to fully avoid any race conditions.

This module makes use of a semaphore to prevent any race condition. When a process or a thread tries to log something, it waits for that semaphore to become available before logging anything. After it becomes available, it will lock that semaphore and log whatever it wants to, and only after it will make it available again. That logging is done to a specified file, which by default is named access.log.

Each log makes use of an indicated log level. What level of logging is to be in fact logged by the server is indicated in the configuration file. There are a total of 4 log levels:

- DEBUG for debug messages; it can be enabled with `--verbose` on server startup; also logs INFO, WARN and ERROR messages
- INFO to provide useful information about what's happening on the server; this the default option; also logs WARN and ERROR messages
- WARN for non critical errors or messages, merely something that should be known; also logs ERROR messages
- ERROR for any critical errors that can be logged; these types of messages are always logged by the server.

There are 2 different methods for logging, one to log requests and one to log any message. The one that logs messages is the generic method. It needs a message to be logged and a log level. When logging becomes available, that message is logged with the indicated level and with a timestamp. The other method logs requests using the Apache Combined Log Format and its log level is always INFO, which is why that level is the default one.
Here is an example of what these logs look like:
Log of request: `127.0.0.1 - - [09/Dec/2025:15:44:48 +0000] "GET ../www/index.html HTTP/1.1" 200 1038`
Log of a message: `[INFO] [09/Dec/2025:15:44:48 +0000] - Worker hit cache for ../www/index.html`

2.6 HTTP Keep-Alive (Optional Feature)

The HTTP Keep-Alive feature allows a client to reuse a single TCP connection for multiple HTTP requests instead of opening a new connection for each request. Before the implementation of this feature, one connection would trigger the four workers, one for each request (our index.html has four files, one file equals one request). Now, one single worker is able to handle all the requests himself, leaving the other workers available to handle other connections.
This also reduces connection overhead, improves latency and decreases server resource usage.

When a client request includes the Connection: keep-alive header, the server keeps the socket open after sending the response. The worker continues listening on the same socket for additional requests until either the client closes the connection or a server-defined timeout expires (we chose 5 seconds of inactivity). Proper timeout and error handling are crucial to avoid hanging connections.

2.7. Range Requests (Optional Feature)

The range requests feature implements the ability for the server to parse and respond to status code 206, which indicates a Partial Content response. Having these types of requests is essential for video streaming to HTML from a local file or for file downloads, as they break the request into more manageable chunks.

The internal structure of the HTTP request was changed to include range start and end, as well as a flag that indicates if the request is in fact ranged. If one of those requests is received, the HTTP parser indicates only which section of bytes to send back, and does so until the full requested file is passed. The last of these requests has status 200 and not 206, indicating that the request was fully completed.

To trigger these types of requests, the easiest way is to request a video file. Whether or not a range request is passed to the server is determined by the web browser, and not the server itself.

2.8. HTTPS/SSL Support (Optional Feature)

The SSL support feature secures communication between the server and client by enabling HTTPS, which provides an encrypted channel. Modern HTTPS uses TLS (often managed through OpenSSL), but the term “SSL” is still commonly used.

Enabling this feature required changing how the server receives requests and sends responses. These operations are now handled through an SSL object, which takes care of encrypting outgoing data and decrypting incoming data. As a result, the server uses OpenSSL-specific functions for reading and writing data.

To establish an encrypted connection, two files were created: a certificate and a private key. The certificate contains the server’s public key and identity information, while the private key is used during the TLS handshake to securely negotiate encryption keys. Because both files are self-signed rather than issued by a trusted Certificate Authority, all web browsers will display a warning before allowing the connection.

2.9. Real-Time Web Dashboard (Optional Feature)

The Real-Time Web Dashboard provides a visual interface for monitoring server statistics and performance metrics in real-time. It allows to:

- Track the number of active connections;
- Monitor responses status code;
- Total requests made;
- Total bytes transferred.

This feature enhances usability, monitoring, and debugging capabilities without accessing the server logs manually, using cards and charts to enhance the user experience.

Statistics are read from the shared memory segment and are sent in JSON format using Server-Sent Events (SSE) every second.

It should be noted that LLM models were used to support the implementation of this feature. They assisted in generating the front-end code and in recalling JSON parsing concepts previously covered in the course's earlier curricular unit, *Introdução a Tecnologias Web.*



3. Performance Tests

Performance testing was conducted to evaluate the server's behavior under various load conditions and to measure its efficiency in handling concurrent client connections. These tests aim to assess key metrics such as response time, and resource utilization, as well as to identify potential bottlenecks in the system.

The primary objectives of these tests are:

- To verify that the master-worker architecture and thread pools maintain stable operation under high concurrency;
- To measure the effectiveness of synchronization mechanisms, such as semaphores, in ensuring data consistency without significantly impacting performance;
- To evaluate optional features, such as HTTPS, HTTP Keep-Alive and the Real-Time Web Dashboard, and their impact on server responsiveness.

By analyzing the results of these tests, we can ensure that the server meets the desired performance requirements and identify areas for future optimization.

Concurrency tests were conducted with the ab terminal command, which allows sending the necessary connections. Please be noted that we used the ab keep-alive flag (-k) in order for the tests to be according to our features. In this case, a template for the command would be ab -k -n <number of requests> -c <number of concurrent connections> https://localhost:8080/index.html.

First of all, we tested the ability to handle multiple requests, but with a single concurrent connection, in order to evaluate the server's base response speed. This test concluded with a 0.075ms (!) response time per request which is at least, exceptionally fast.

```
Concurrency Level:      1
Time taken for tests:   0.748 seconds
Complete requests:      10000
Failed requests:        0
Keep-Alive requests:    10000
Total transferred:      10380000 bytes
HTML transferred:       9160000 bytes
Requests per second:    13377.85 [#/sec] (mean)
Time per request:       0.075 [ms] (mean)
Time per request:       0.075 [ms] (mean, across all concurrent requests)
Transfer rate:          13560.76 [Kbytes/sec] received
```

In the second test, we ran 10000 requests with 100 concurrent operations, that gave us the following result:

```
Concurrency Level:      100
Time taken for tests:   0.903 seconds
Complete requests:      10000
Failed requests:        0
Keep-Alive requests:    10000
Total transferred:      10380000 bytes
HTML transferred:       9160000 bytes
Requests per second:    11078.61 [#/sec] (mean)
Time per request:       9.026 [ms] (mean)
Time per request:       0.090 [ms] (mean, across all concurrent requests)
Transfer rate:          11230.08 [Kbytes/sec] received
```

```
Percentage of the requests served within a certain time (ms)
  50%      3
  66%      3
  75%      3
  80%      4
  90%      5
  95%      6
  98%      8
  99%      9
 100%     70 (longest request)
```

In order to take our server to its limit, we tried to figure out, by hand, the maximum number of concurrent operations it can handle while completing every request successfully, which concluded to be 440 concurrent requests. After that number, there would be some failed requests, although they were never more than 10% of all.

We are sorry that this issue was found late into the project development and due to lack of time we could not find neither its root cause nor a fix.

```
Concurrency Level:      440
Time taken for tests:   0.966 seconds
Complete requests:      10000
Failed requests:        0
Keep-Alive requests:    10000
Total transferred:      10380000 bytes
HTML transferred:       9160000 bytes
Requests per second:    10351.60 [#/sec] (mean)
Time per request:       42.505 [ms] (mean)
Time per request:       0.097 [ms] (mean, across all concurrent requests)
Transfer rate:          10493.13 [Kbytes/sec] received
```

```
Concurrency Level:      1000
Time taken for tests:   1.129 seconds
Complete requests:      10000
Failed requests:        970
   (Connect: 0, Receive: 0, Length: 970, Exceptions: 0)
Non-2xx responses:      970
Keep-Alive requests:    9031
Total transferred:      9498338 bytes
HTML transferred:       8303436 bytes
Requests per second:    8859.20 [#/sec] (mean)
Time per request:       112.877 [ms] (mean)
Time per request:       0.113 [ms] (mean, across all concurrent requests)
Transfer rate:          8217.55 [Kbytes/sec] received
```

4. Conclusion

This document provided a technical overview of the project, detailing the implementation of the core and optional server features The system uses a master-worker architecture with thread pools and shared memory synchronization to ensure concurrent, reliable, and scalable operations. The challenges encountered during this project provided valuable learning opportunities in concurrent programming, enabling us to develop essential skills that would have not developed otherwise.

Optional features improve performance, security and usability, while the graceful shutdown mechanism ensures proper cleanup of resources and termination of worker processes. Overall, the project demonstrates effective concurrent programming and robust server design.